TUTORIAL – 10: Automation & Regression Analysis (Python – PyCharm)

Aim

To automate the testing of the Login Module using **Python** in **PyCharm** and perform **Regression Testing** after modifying the application to ensure that existing functionality remains unchanged.

Description

Automation Testing is a software testing technique where test cases are executed automatically using scripts instead of manually.

Python is widely used for automation because it is simple, readable, and supports many testing libraries such as:

- unittest
- pytest
- Selenium
- Robot Framework

In this practical, students will use **Python's unittest framework** to automate the Login Module.

After executing automated test cases, Regression Testing is performed to verify that software modifications have not affected existing functionality.

Problem Statement

Develop an automated test script in Python to verify the Login Module of the Student Management System using multiple input conditions.

Execute regression test cases after modifying the application and verify that all existing functionalities work correctly.

How to Perform This Practical

Step 1

Open **PyCharm**.

Create a project

AutomationTesting

Step 2
Create two Python files
login.py

and
test_login.py

Step 3
Write Login Program
Example

```
def login(username, password):

    if username == "admin" and password == "admin123":
        return "Login Successful"

    else:
        return "Invalid Username or Password"
```

Save the file.

Step 4
Create Automated Test Script

```
import unittest
from login import login

class LoginTest(unittest.TestCase):

    def test_valid_login(self):
        self.assertEqual(login("admin","admin123"),
                         "Login Successful")

    def test_invalid_username(self):
        self.assertEqual(login("student","admin123"),
                         "Invalid Username or Password")
```

```
    def test_invalid_password(self):
        self.assertEqual(login("admin","12345"),
                         "Invalid Username or Password")

    def test_blank_username(self):
        self.assertEqual(login("","admin123"),
                         "Invalid Username or Password")

    def test_blank_password(self):
        self.assertEqual(login("admin",""),
                         "Invalid Username or Password")

if __name__=="__main__":
    unittest.main()
```

Step 5
Run the program
Right Click
test_login.py

↓
Run

Step 6
Observe the Result
If all tests pass
OK

Ran 5 tests

PASSED

Step 7
Modify Login Program
Example
Change
admin123

to
admin321

Run all test cases again.
Regression testing verifies whether previous functionality is affected.

Step 8
Record
Expected Result
Actual Result
Status

Implementation Details
The Login Module is automated using Python's **unittest** framework. Automated scripts verify valid and invalid login scenarios. After modifying the application, regression testing is executed to ensure previously working functionalities remain correct.

Parameter Details

Parameter	Details
Project Name	Student Management System
Application/Software	Login Module
Module Name	User Authentication
SDLC Model	Waterfall
Programming Language	Python

IDE/Tool Used	PyCharm
Input/Test Data	admin/admin123, admin/12345, student/admin123

Source Code

login.py

```
def login(username,password):

    if username=="admin" and password=="admin123":
        return "Login Successful"

    return "Invalid Username or Password"
```

test_login.py

```
import unittest
from login import login

class LoginTest(unittest.TestCase):

    def test_valid_login(self):
        self.assertEqual(login("admin","admin123"),
                         "Login Successful")

    def test_invalid_username(self):
        self.assertEqual(login("student","admin123"),
                         "Invalid Username or Password")

    def test_invalid_password(self):
        self.assertEqual(login("admin","123"),
                         "Invalid Username or Password")

    def test_blank_username(self):
```

```
        self.assertEqual(login("","admin123"),
                         "Invalid Username or Password")

    def test_blank_password(self):
        self.assertEqual(login("admin",""),
                         "Invalid Username or Password")

if __name__=="__main__":
    unittest.main()
```

Module Analysis Table

Module Name	Functionality Critical	Frequently Changed	Suitable for Automation	Reason
Login	Yes	No	Yes	Executed repeatedly
Registration	Yes	No	Yes	Validation testing
Student Details	Yes	Yes	No	Frequent modifications
Search	Yes	No	Yes	Stable functionality
Logout	No	No	Yes	Simple operation

Automation Candidate Identification

Test Case ID	Functionality	Automation Recommended	Reason	Automation Tool
TC001	Login	Yes	Frequently executed	Python unittest

TC002	Registration	Yes	Stable Module	Python unittest
TC003	Search	Yes	Repetitive testing	Python unittest
TC004	Student Details	No	Frequently changing	Manual Testing

Test Case Detailed

Field	Details
Test Case ID	TC001
Requirement ID	R001
Module Name	Login
Feature Under Test	User Login
Test Scenario	Valid Login
Test Objective	Verify successful login
Test Type	Regression Testing
Test Technique	Black Box Testing
Priority	High
Severity	Critical
Preconditions	Application is running
Postconditions	User logged in

Test Environment	Windows 10
Browser	Not Applicable
Test Data	admin/admin123
Expected Result	Login Successful
Actual Result	Login Successful
Status	Pass
Executed By	Student
Execution Date	DD/MM/YYYY

Test Case Execution

Step	Test Steps	Input	Expected Result	Actual Result	Status
1	Run login.py	-	Program executes	Program executes	Pass
2	Run test_login.py	-	Test starts	Test starts	Pass
3	Execute Valid Login	admin/admin123	Login Successful	Login Successful	Pass
4	Execute Invalid Login	admin/123	Invalid Login	Invalid Login	Pass
5	Run Regression Tests	-	All previous tests pass	All previous tests pass	Pass

Regression Test Suite

Regression Test ID	Module	Scenario	Expected Result	Status
RT001	Login	Valid Login	Pass	Pass
RT002	Login	Invalid Username	Pass	Pass
RT003	Login	Invalid Password	Pass	Pass
RT004	Login	Blank Fields	Pass	Pass

Automation Feasibility Report

Criteria	Observation
Repetitive Test Cases	Yes
Stable Functionality	Yes
High Business Priority	Yes
Frequently Executed	Yes
Suitable Automation Tool	Python unittest
Recommendation	Automate Login Module

Regression Testing Summary

Metric	Value
Total Modules Analyzed	4

Automation Candidates	3
Regression Test Cases	4
Modules Excluded	Student Details
Recommendation	Automate stable modules using Python unittest

Observation

Sr.No	Observation	Remarks
1	Automation script executed successfully	Pass
2	All automated test cases passed	Pass
3	Regression testing verified previous functionality	Pass
4	Python unittest generated accurate results	Pass
5	Login module suitable for automation	Completed

Output

```
.....
----------------------------------------------------------------------
Ran 5 tests in 0.002s

OK

==========================================
Automation Testing Completed
```

Regression Testing Completed

Total Test Cases : 5
Passed : 5
Failed : 0

Overall Result : PASS
==========